llm-smart-router

A multi-model AI agent orchestrator with cross-subscription CLI hybrid mode

Final Project Report June Gu 06-642 · Spring 2026 Carnegie Mellon University

GitHub: https://github.com/junegu-glitch/llm-smart-router

---

1. The Idea

For this final project, I extended `llm-smart-router` from a CLI-only tool into a multi-surface system that coordinates multiple LLM subscriptions in a single workflow. The goal was practical: many of us already pay for several $20/month plans (Claude, Gemini, Codex), and we burn through their token quotas faster than expected. I wanted one tool that could spread work across those subscriptions intelligently, without losing workflow continuity.

The main user is still a developer or technical user who works across multiple AI systems. The new addition is a non-terminal user — somebody who wants the same routing and team workflows from a browser instead of a shell.

The Inspiration

Anthropic has demonstrated an "agent team" pattern where you assign distinct roles (CEO, designer, finance manager) to multiple agents and let them collaborate on a problem. That pattern is compelling because each teammate has its own fresh context window, which avoids the context-accumulation problem that makes long single-session agents progressively dumber.

The problem is that running this kind of team entirely within one LLM subscription drains tokens very quickly. Switching to a different provider mid-project loses workflow continuity and costs even more tokens to reload context.

That led to the core question of this project:

> *Can we form an agent team that spans different LLM services, so that we can use tokens efficiently across subscriptions without losing workflow continuity?*

What I Built

Two main capabilities:

- **Smart routing** — a lightweight classifier categorizes each request into one of seven task types and selects the highest-ranked available model.
- **Multi-model team mode** — a leader model plans a team, assigns roles to teammate agents, runs them in parallel, and synthesizes a final report.

The differentiating feature is `--use-cli` : team runs route through locally installed subscription CLIs (`claude` , `gemini` , `codex`) using `child_process.spawn` , which means a full team run costs **$0** in API tokens when the relevant CLIs are installed.

What Is Different From Project-2

Category	Project-2	Final Project
Primary problem	Smart routing + team mode (CLI only)	Same + zero-cost team via subscription CLIs

Audience	Developers comfortable with terminal	+ non-terminal users via web UI
Interface	TypeScript CLI	CLI + optional Next.js web UI
Cost model	API tokens per call	$0 with `--use-cli` (subscription CLIs)
Test count	154	167
CI/CD	None	GitHub Actions (Node 20 + 22)
Persistent context	Per-session only	Wiki system across sessions

The final project keeps the Project-2 CLI core unchanged and layers four new capabilities on top: subscription-CLI hybrid mode, web UI, CI/CD, and a session-persistent wiki for working with Claude Code.

---

2. The Plan

Before writing code, I planned the final-project additions on top of the Project-2 CLI as four separable layers, each shippable on its own.

Planned Command Surface

- `smart-router "<message>"` — single routed query (existing)
- `smart-router team run --use-cli "..."` — zero-cost team run (new)
- `smart-router team presets` / `team review <id>` — preset and replay (existing)
- `smart-router serve` — start the optional web UI locally (new)
- `smart-router config set <provider> <key>` — BYOK key management (existing)

Planned Inputs & Outputs

Inputs: a natural-language task or prompt, optional git diff context, optional file context, configured provider credentials, and detected subscription-CLI binaries. Outputs: a routed single-model response, a planned multi-teammate run with status and cost reporting, a synthesized final report, and saved team sessions for later review.

Planned Architecture (Two Layers)

The first layer is a seven-category router. Every request is classified into `coding`, `writing`, `math_reasoning`, `analysis`, `image_multimodal`, `large_document`, or `general`, and routed to the highest-ranked available model for that category. Each category has a ranked fallback chain.

The second layer is the team orchestrator. A leader model receives the request, decides how many teammates to spawn, what specialist role each plays, and which model fits each role. Teammates run in parallel. The leader then reads all outputs and writes the final synthesis.

Both the CLI and the web UI use the same `src/lib/` core, so any improvement to the routing logic applies equally to both surfaces.

Planned Libraries

- Node.js + TypeScript
- `commander` for the CLI
- `vitest` for automated tests

- `child_process.spawn` for subscription-CLI integration
- Next.js + Supabase for the optional web UI
- AES-256-GCM for local API-key encryption

Planned Edge Cases

- No API keys configured
- Provider authentication failures (401)
- Rate limit responses (429)
- Request timeouts (120-second per-teammate cap)
- Subscription CLI binary not installed or not authenticated
- Mixed-type prompts that do not map cleanly to a single category
- Teammate failure during a multi-step run, with partial synthesis still useful

---

3. How You Built It

Initial Prompt

The Project-2 baseline already existed. The final project started from a broader prompt that asked Claude Code to extend the existing CLI into a multi-surface system:

> *Extend `llm-smart-router` with three new layers: a `--use-cli` mode that routes each teammate through a locally installed subscription CLI binary instead of the API, an optional Next.js web UI that uses the same routing core, and a GitHub Actions CI pipeline. Preserve the existing CLI behavior and tests.*

Key Follow-Up Prompts

After the initial scaffold, I guided Claude Code toward a specific shape:

- Keep the CLI as the primary interface; the web UI is strictly optional
- Detect three subscription CLIs (`claude` , `gemini` , `codex`) at startup
- Route teammate calls through `child_process.spawn` only when the binary is available; otherwise fall back to API mode
- Add a `viaCLI: true` flag on results so the cost rollup can report $0
- Use Supabase only for sync of the encrypted blob, never for the plaintext key
- Maintain a project wiki (`wiki/index.md` , `/ingest` , `/query` , `/lint`) so new sessions can reload full project context

Where I Corrected the AI

Three corrections mattered most.

First, the subscription CLI output parsing. The `--use-cli` mode requires parsing the stdout of three different binaries, each with a different output format and streaming behavior. Claude Code initially produced parsing logic that looked correct but failed silently on certain output shapes. Fixing this required reading the actual subprocess output and tracing the parser by hand.

Second, the cost-rollup logic. An earlier version of the team result aggregator did not respect the `viaCLI: true` flag in cost summation, so `--use-cli` runs were reported with non-zero cost even when no API tokens were consumed. I pushed the implementation toward strict separation: `cost = 0` when `viaCLI: true` , never an estimate.

Third, the project framing. Claude Code's initial framing for the final project was "API key management with a web UI," which is not the differentiating story. I steered the framing toward the more specific value

proposition: zero-cost team mode, cross-model orchestration, and the subscription-CLI hybrid as the hero feature.

Multi-Model Collaborative Workflow

One less obvious decision was to use multiple different models during development itself, not just as the target of the router.

Before committing to the project direction, I asked Gemini to deep-research whether the cross-subscription CLI hybrid idea was novel. After about 20 minutes it returned a thorough competitive landscape. There were adjacent projects, but the specific combination of subscription CLI hybrid mode + smart routing + team orchestration still appeared novel. That cleared the way to keep working.

For significant technical decisions, I developed a multi-model loop: I would ultra-plan the approach with Claude, send the plan to Codex for adversarial review, bring the review back to Claude to discuss whether the critiques were valid, and separately discuss my own intuitions with Gemini. This was not a formal pipeline — it was conversational and iterative — but it consistently surfaced perspectives I would not have reached in a single-model session.

The key observation: different models have genuinely distinct reasonable perspectives. Claude tends to think from correctness and safety first; Codex tends to think from structure and planning; Gemini tends to take a broader systems view. Using all three produced better designs than any one alone.

Why a Web UI

I have been using the terminal since joining Kitchin lab, but before that I had no command-line experience. Building this tool for researchers and students who are in the same position I was — technically capable, but not yet comfortable in a terminal — motivated the optional web UI. The goal was an interface that could be used intuitively by anyone who had already interacted with a chat-based AI product, without requiring CLI knowledge. The CLI and the web UI share the same `src/lib/` routing and orchestration core, so any improvement to the router applies equally to both surfaces.

---

4. Evidence It Works

All commands below were run from the standalone repository at `/Users/junemog/Documents/GitHub/llm-smart-router`. The repository has 167 automated tests across 14 test files, GitHub Actions CI on Node 20 + 22, an optional Next.js web UI, and four runnable examples in `examples/`.

Continuous Integration

The repository runs a GitHub Actions pipeline on every push and pull request to `main`. The matrix covers Node.js 20 and 22.

COMMAND

```
# .github/workflows/ci.yml
- npm ci
- npm run build:cli
- npm test
- npm run lint
```

OBSERVED OUTPUT (EXCERPT)

```
✓ build:cli   passed (Node 20)
✓ test        167 passed (Node 20)
✓ lint        clean (Node 20)
✓ build:cli   passed (Node 22)
✓ test        167 passed (Node 22)
✓ lint        clean (Node 22)
```

Both matrix legs pass at the current head commit. The CI badge in the README links to the live Actions run history.

Automated Tests

COMMAND

```
npm test
```

OBSERVED OUTPUT (EXCERPT)

```
Test Files  14 passed (14)
     Tests  167 passed (167)
  Duration  423ms
```

Per-file breakdown of the 167 tests:

File	Scope	Tests
`router.test.ts`	Task classification + model selection	22
`llm-client.test.ts`	Hybrid CLI/API routing	13
`llm-client-cli.test.ts`	`callLLM` CLI hybrid branch	8
`cli-provider.test.ts`	CLI detection + subprocess execution	32
`models.test.ts`	Model catalog and pricing	17
`team.test.ts`	Full team orchestration flow	8
`leader.test.ts`	Plan generation and synthesis	6
`teammate.test.ts`	Parallel execution and fallback	13
`presets.test.ts`	Preset configuration	23
`team-session.test.ts`	Session save/load/list	8
`team-use-cli.test.ts`	Team run $0 cost rollup	5
`config.test.ts`	API key storage and encryption	4
`git-context.test.ts`	Git diff and file context parsing	5
`output.test.ts`	Terminal formatting	3

This is an increase of 13 tests over Project-2 (154 -> 167), with the new tests targeting the `--use-cli` hybrid branch and the team-mode $0 cost rollup.

Help Output

COMMAND

```
smart-router --help
```

OBSERVED OUTPUT (EXCERPT)

```
Usage: smart-router [options] [command] [message...]

Route your messages to the best AI model

Options:
  -i, --interactive    Start interactive REPL mode
  -m, --model <model>  Force a specific model
  -t, --tier <tier>    Preferred tier: budget, mid, premium
  -v, --verbose        Show routing details
  --use-cli            Use CLI binaries instead of API ($0 cost)

Commands:
  config               Manage API keys and settings
  team [message...]    Multi-model agent team commands
  models               List available models and their status
  serve [options]      Start the web UI locally
```

This confirms the CLI surface is exposed correctly and the new `--use-cli` flag and `serve` command are discoverable from the terminal.

Model Catalog

COMMAND

```
smart-router models
```

OBSERVED OUTPUT (EXCERPT)

```
Available Models:
  ✓ GPT-4.1 Nano          openai     budget   $0.40/1M out
  ✓ DeepSeek V3.2         deepseek   budget   $0.028/1M out
  ✓ Gemini 2.5 Flash      google     budget   $0.60/1M out
  ✓ GPT-5.2               openai     premium  $7.00/1M out
  ✓ Claude Sonnet 4.6     anthropic  premium  $15.00/1M out
```

This confirms the model catalog is enumerated correctly at runtime, with provider availability and pricing tier visible.

Team Presets

COMMAND

```
smart-router team presets
```

OBSERVED OUTPUT (EXCERPT)

```
Available Team Presets:
  code-review   Code Review Team        3-person code review: security, quality, docs
  debug         Debug Team              3-person debug squad: root cause, fix, tests
  explain       Code Explainer Team     2-person: technical deep-dive + beginner
explanation
  refactor      Refactoring Team        3-person: architecture, refactored code,
migration plan
```

This confirms the four preset team configurations from Project-2 still load correctly in the final-project build.

Saved Successful Session

The strongest end-to-end artifact in the repository is a saved team-mode run that completed successfully with valid provider credentials. The session is reopened by partial ID:

COMMAND

```
smart-router team review 22afb8d1
```

OBSERVED OUTPUT (EXCERPT)

```
Team Session Review
Request: Compare the pros and cons of Python vs Rust for building
         a web scraper. I need: 1) A technical comparison, 2) A
         practical code example in each language
Status: done
Cost: $0.093648
Time: 97.9s

Teammates:
✓ Analyst   Claude Sonnet 4.6   $0.031212
✓ Coder     Claude Sonnet 4.6   $0.062310
```

This shows a completed end-to-end team run with the original request, completion status, runtime, cost, and teammate roles. The command also prints the saved synthesis below the metadata.

Zero-Cost Team Run with `--use-cli`

This is the hero feature of the final project. With `--use-cli` enabled, each teammate routes through a locally installed subscription CLI binary rather than billable API tokens.

COMMAND

```
smart-router team run --use-cli "Compare Python vs Rust for building a web scraper"
```

OBSERVED OUTPUT (EXCERPT)

```
Probing CLI binaries…
✓ claude    (Anthropic — Claude Sonnet 4.6)
✓ gemini    (Google — Gemini 2.5 Flash)
✗ codex     (not found)

Planning team…
Leader -> Gemini 2.5 Flash (via CLI)

Spawning 2 teammates (parallel)…
  - Analyst   -> Claude Sonnet 4.6  [CLI · $0]  running...
  - Coder     -> Claude Sonnet 4.6  [CLI · $0]  running...

✓ Analyst    completed  (43s)
✓ Coder      completed  (61s)

----------------------------------------
Total API cost:  $0.00
Routing:         2/2 teammates via CLI  ·  leader via CLI
Duration:        91s
```

All three model calls (leader + 2 teammates) routed through the `claude` and `gemini` subscription binaries. No API tokens were consumed. This is the direct evidence for the differentiating claim of the final project.

Error Handling

COMMAND

```
HOME=/tmp/llm-smart-router-empty smart-router "hello"
```

OBSERVED OUTPUT (EXCERPT)

```
No API keys configured.
Run: smart-router config set <provider> <key>
Example: smart-router config set deepseek sk-xxx
```

This confirms the CLI has a clean failure path for an unconfigured environment and gives the user a direct recovery command instead of a generic stack trace.

---

5. Reflection

What Worked Well When Collaborating With Claude Code?

Claude Code was most effective when the design intent was clear. Once I had a specific shape in mind for a feature — for example, "detect three CLI binaries at startup and route teammate calls through `child_process.spawn` only when the binary is available" — Claude Code produced working scaffolding fast and matched the existing project conventions.

It was also strong at maintaining the wiki system. The `/ingest` `/query` `/lint` slash commands let me feed Claude Code a curated project context at the start of each session, which made multi-session work feel continuous.

What Did Not Work, and Where Did the AI Fall Short?

Claude Code was weakest on provider-specific edge cases. Every provider integration required at least one explicit correction for an error path the AI had been too optimistic about. The subscription-CLI parsers were the most extreme case: the AI produced code that looked correct and passed shallow tests, but failed silently on real subprocess output until I traced the parsing manually.

In very long sessions Claude Code would sometimes drift back to earlier patterns I had explicitly asked it to abandon. The wiki system mitigated this across sessions, but did not solve it within a single long session.

How Did My Use of Claude Code Evolve Across the Semester?

At the beginning of the course, my mental model was: AI writes a draft, I fix it. I gave Claude Code specific prompts, accepted what it produced, and manually corrected problems I found.

By Project 2, I started planning thoroughly before executing. I would discuss with Claude whether my overall approach made sense before writing any code. The shift was from "generate code" to "validate ideas." I also learned to give Claude Code very precise corrections rather than vague ones.

For the final project, the workflow had three new pieces: plan mode before any significant change, a persistent wiki so each new session started with full project context, and the use of multiple different models for different roles in development itself (planning with Claude, review with Codex, perspective checks with Gemini).

The meta-moment of the project is deliberate: I used Claude Code to build a tool that runs Claude, Gemini, and Codex as parallel subprocesses. The orchestrator was built by an agent. The agents it orchestrates include the agent that built it.

How Did You Troubleshoot Problems?

I kept moving from broad product prompts toward direct verification:

- Checking `smart-router --help` and `smart-router models` before assuming the CLI surface was correct
- Reading actual subprocess stdout when CLI parsing was off, instead of trusting AI-written parser comments
- Reopening saved session artifacts when fresh runs were blocked by quota or authentication issues
- Running `npm test` and `npm run lint` after every meaningful change rather than at the end
- Using GitHub Actions as a third-party verifier on a clean environment

That pattern made it easier to catch the difference between "the design seems reasonable" and "the CLI actually behaves correctly in a real terminal."

What Are the Current Limitations?

- **Provider setup friction** — BYOK requires manual per-provider key configuration. A guided `smart-router auth` wizard is the obvious next step.
- **Routing edge cases** — Mixed-type prompts (math + code in one request) do not map cleanly to a single category. A more granular or ensemble classifier would help.
- **Cross-model verification** — The README describes a Claude -> Gemini -> Codex -> Claude judge pipeline as a natural extension, but this preset is not yet implemented; it requires chained rather than parallel execution.
- **Maintenance** — TypeScript is not yet a language I can debug fluently without AI assistance. I was able to design the system, specify behavior, review outputs, write tests, and correct failure modes — but a future provider API change or subtle subprocess parsing regression may still require AI-

assisted debugging to resolve. This is both a project limitation and a broader lesson from the semester: AI coding agents can let a developer build beyond their current language fluency, but long-term maintainability still depends on the human collaborator deepening their own debugging understanding.

Would You Actually Use This Tool? What Would It Take To Make It Production-Ready?

Yes. I already switch between Claude, Gemini, and Codex daily, and I would rather have one tool that spreads work across my existing subscriptions than keep manually choosing one model at a time.

To make it more production-ready, I would add:

- A guided `smart-router auth` setup wizard for BYOK keys
- A chained cross-model verification preset (Claude -> Gemini -> Codex)
- Stronger subscription-CLI health checks before a team run starts
- Broader end-to-end tests against mocked provider failures and CLI output shape regressions